

Program 7:

Deploy an Automated Build Pipeline using Docker Hub

12/12/2025

Project Structure:

```
prog7
|- nginx-deployment.yaml
```

Steps for Execution:

Step 1: Check for prerequisites,

```
> minikube version
> kubectl version --client
> docker --version
```

Step 2: Create a project folder as prog7 and move into it

```
> mkdir prog7
> cd prog7/
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~$ cd devops_lab/
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab$ mkdir prog7
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab$ cd prog7/
```

Step 3: Start the minikube and check the status

```
> minikube start --driver=docker
> minikube status
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ minikube version
kubectl version --client
docker --version
minikube version: v1.37.0
commit: 65318f4cfff9c12cc87ec9eb8f4cdd57b25047f3
Client Version: v1.31.0
Kustomize Version: v5.4.2
Docker version 29.0.4, build 3247a5a
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ minikube start --driver=docker
🐹 minikube v1.37.0 on Ubuntu 24.04
🔧 Using the docker driver based on existing profile
👉 Starting "minikube" primary control-plane node in "minikube" cluster
📦 Pulling base image v0.0.48 ...
🔄 Restarting existing docker container for "minikube" ...
🔧 Preparing Kubernetes v1.34.0 on Docker 28.4.0 ...
   ▪ apiserver.enable-admission-plugins=
🔍 Verifying Kubernetes components...
   ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟 Enabled addons: default-storageclass, storage-provisioner

! /usr/local/bin/kubectl is version 1.31.0, which may have incompatibilities with Kubernetes 1.34.0.
   ▪ Want kubectl v1.34.0? Try 'minikube kubectl -- get pods -A'
🏠 Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ minikube status
minikube
type: Control Plane
host: Running
kubelet: Running
apiserver: Running
kubeconfig: Configured
```

Step 4: Create a nginx-deployment.yaml file

```
> nano nginx-deployment.yaml
```

Write the following code inside it

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ nano nginx-deployment.yaml
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ cat nginx-deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          ports:
            - containerPort: 80
```

Step 5: After writing the yaml file apply the deployment where we have created the yaml file
 > kubectl apply -f nginx-deployment.yaml

Step 6: Check and get the deployments and pods in that directory
 > kubectl get deployments
 > kubectl get pods

Step 7: Now the deployments will be exposed as a service by using the following command
 > kubectl expose deployment nginx-deployment \
 --type=NodePort \
 --port=80

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ kubectl apply -f nginx-deployment.yaml
deployment.apps/nginx-deployment created
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ kubectl get deployments
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
nginx-deployment    0/2     2             0           6s
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-deployment-6f9664446b-g9dg8  1/1     Running   0           16s
nginx-deployment-6f9664446b-nsmtk  1/1     Running   0           16s
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ kubectl expose deployment nginx-deployment \
--type=NodePort \
--port=80
service/nginx-deployment exposed
```

Step 8: We can verify the service by the command
 > kubectl get svc

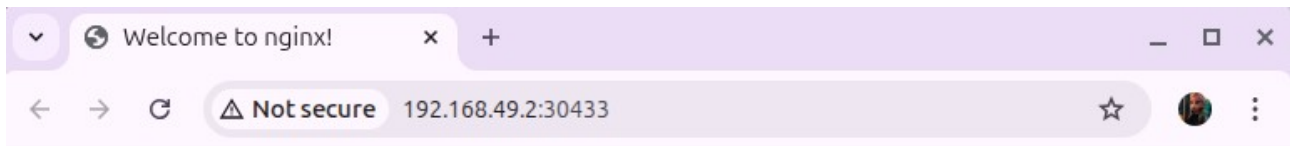
Step 9: Now we can access the web app using the minikube by the command,
 > minikube service nginx-deployment

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ kubectl get svc
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)          AGE
kubernetes          ClusterIP   10.96.0.1       <none>       443/TCP          116m
nginx-deployment    NodePort    10.110.25.103   <none>       80:30433/TCP     6s
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$ minikube service nginx-deployment
```

NAMESPACE	NAME	TARGET PORT	URL
default	nginx-deployment	80	http://192.168.49.2:30433

 Opening service default/nginx-deployment in default browser...

Step 10: The above command opens the browser automatically where we can see the web application



Welcome to nginx!

If you see this page, the nginx web server is successfully installed and working. Further configuration is required.

For online documentation and support please refer to nginx.org.
Commercial support is available at nginx.com.

Thank you for using nginx.

Step 11: Now delete all the pods, services and deployments

```
> kubectl delete pods --all
```

```
> kubectl delete svc --all
```

```
> kubectl delete deployments --all
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$
```

```
kubectl delete pods --all
```

```
pod "nginx-deployment-6f9664446b-g9dg8" deleted
```

```
pod "nginx-deployment-6f9664446b-nsmk" deleted
```

```
pod "nodejs-pod" deleted
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$
```

```
kubectl delete svc --all
```

```
service "kubernetes" deleted
```

```
service "nginx-deployment" deleted
```

```
1RV24MC030_CHINMAYI_M_H@chinnu-HP-Pavilion-Laptop-15-eg3xxx:~/devops_lab/prog7$
```

```
kubectl delete deployments --all
```

```
deployment.apps "nginx-deployment" deleted
```